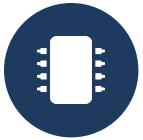
Do the Numbers Hold on Real Hardware?

A reproducibility and measurement audit of a low-cost hybrid ECC authentication protocol for IoT

Jay Kerkar · Chaitanya Kelkar · Chirag Poornamath · Yash Kolekar
Project guide: Dr. Shanta Sondur

Tiny devices need cheap authentication



The device is weak

Little memory and a slow processor, so heavy cryptography is costly.



It must prove who it is

Device and server authenticate each other before any data is exchanged.



The base paper claims savings

Al-Rasheed et al., IEEE TCE 72(2), May 2026, proposes a lighter protocol.

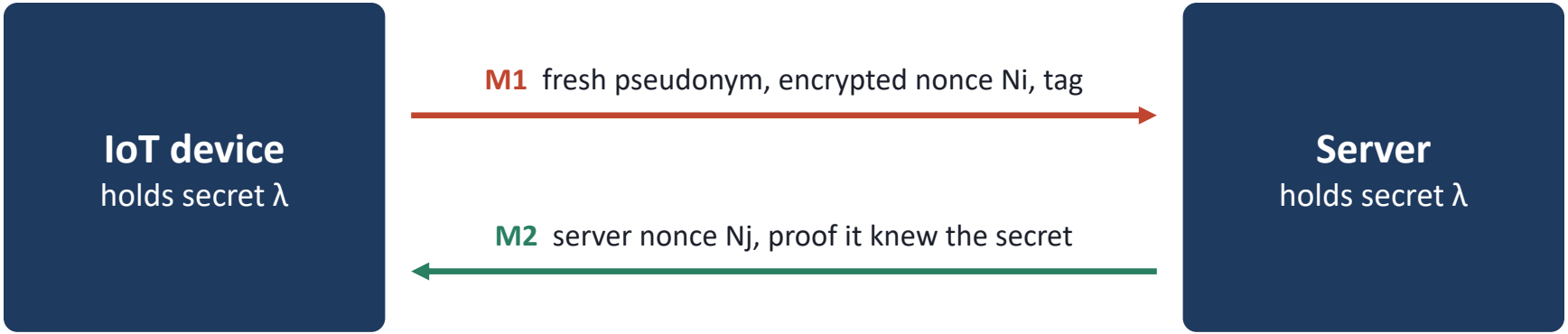
Claimed reduction vs. prior schemes

17% communication

21% processing

12% storage

The protocol: two messages, one shared key



Both sides then compute the same session key

$$K = KDF(k^* \| Ni \| Nj \| t)$$

Where is the ECC?
Only offline, when a trusted authority issues secrets. Online: hashes, AES-GCM, HKDF.

How the paper argued it is cheap

$$cost = n_hash \times T_H + n_enc \times T_SE/D + n_ECC \times T_ECM$$

T_H (assumed)

0.32 ms

one SHA-1 hash

T_SE/D (assumed)

5.6 ms

one symmetric encrypt / decrypt

T_ECM (assumed)

17.1 ms

one ECC scalar multiplication

These prices were assumed, not measured. The OMNeT++ simulation behind Fig. 1 has no published code, scenario or data, so it cannot be re-run.

What we did

1 Audit

Checked every table and figure against the paper's own text.

2 Implement

Python reference plus an independent C core, equation by equation.

3 Measure

ESP8266 at 80 and 160 MHz, cloud Graviton2, laptop. Median of 100 runs.

4 Recompute

Re-ran the paper's own cost model with measured prices.

108 tests pass. The C device on the ESP8266 and the Python server complete 20 of 20 handshakes with identical session keys; replays are rejected.

Finding 1: numbers don't reproduce

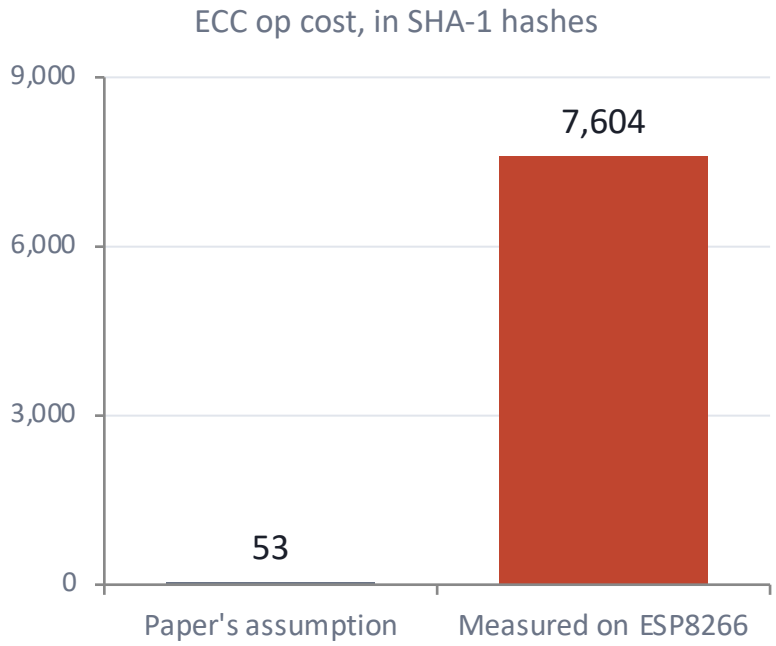
0 of 9

ways of computing 17 / 21 / 12 % from the paper's own tables give those values.

What is reported	Table IV	Text
Messages	**4**	**2**
Bits	**1,024**	**264**

- Fig. 1 is captioned “processing”, but its axis shows communication overhead.
- One competitor row has its device and server counts swapped.
- The simulation cannot be re-run, so Fig. 1 cannot be checked.

Finding 2: the assumed prices are wrong



Operation	Assumed	Measured	Off by
ECC P-256	17.1 ms	276 ms	**16× slower**
SHA-1	0.32 ms	0.036 ms	**8.8× faster**
AES-GCM	5.6 ms	0.5–1.2 ms	**5–12× faster**

Same C source on every platform; ratios are identical at 80 and 160 MHz and on a second board. Medians of 100 runs.

ECC is far costlier, and symmetric crypto far cheaper, than the paper's cost model assumes.

Finding 3: the ranking cuts both ways

Versus ECC-based schemes

60–2,520×

cheaper than the paper claims. The scheme wins, by far more than reported.

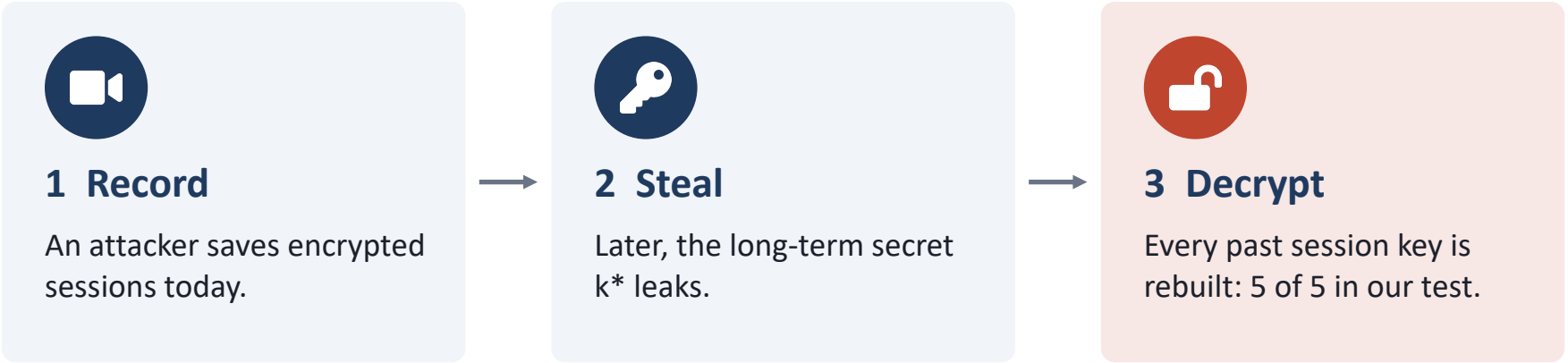
Versus hash-only schemes

3.8–13.7×

more expensive, once the session-key derivation is counted. The paper says the opposite.

Why: the paper's own Eq. 16 needs a key-derivation step (HKDF, 629 µs, about 17 hashes) that its cost table leaves out.

Finding 4: no forward secrecy

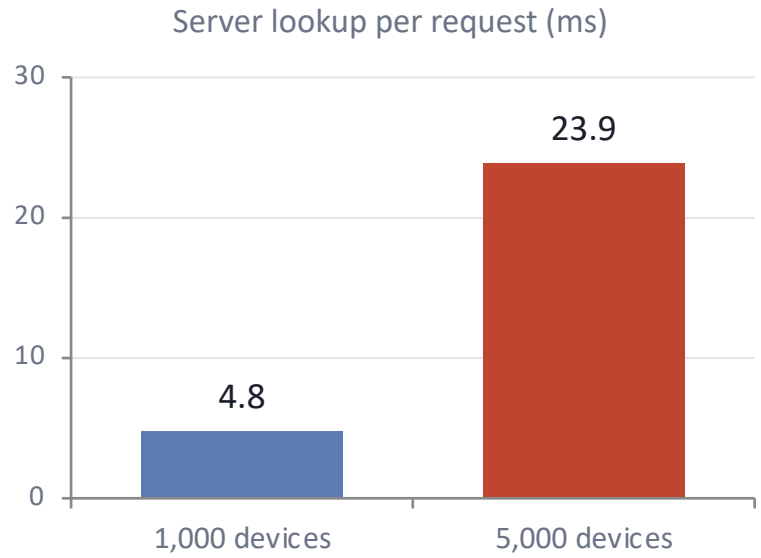


Why: K = KDF(k* ‖ Ni ‖ Nj ‖ t) uses no ephemeral secret, so k* plus recorded traffic rebuilds K. Fixing it needs a fresh key exchange per session, which is an ECC operation (276 ms on our chip).

Finding 5: the server does a hidden loop

```
for device in all_devices:
    for t in epoch_window:
        if H2(device.secret, t) == pid:
            return device
```

Pseudonyms change every message, so the server cannot look one up. It tries every device, and the cost grows with the fleet. The paper's table treats server cost as a constant.



A precomputed table removes the loop at the price of memory (about 416 KB in our setup).

It does run on a real microcontroller

5.26 ms

device compute per handshake

1.67 build + 3.59 verify

216 B

first message (M1)

52 ms over Wi-Fi

276 ms

one ECC multiplication

on the same chip

≈ 52×

that ECC op vs. the whole handshake

276 ms ÷ 5.26 ms

The protocol itself is genuinely light. The paper's weakness is its evidence and its constants, not the idea.

What we show, and what we don't claim

We show

- Headline savings are not derivable from the published tables
- Cost constants are off by 16× and 5–12×
- The ranking depends on counting the KDF
- No forward secrecy; server cost grows with devices

We do not claim

- That the authors' simulation is wrong, only that it cannot be reproduced
- Results beyond one ESP8266 and one crypto library
- That we re-implemented the rivals; two table rows were matched via secondary sources only
- A formal security proof; the authors were not contacted

Take-away

The savings cannot be reproduced from what was published. On real hardware the scheme beats ECC-heavy rivals by a wide margin, loses to hash-only ones, has no forward secrecy, and its server cost grows with the fleet.

Base paper: Al-Rasheed et al., IEEE Trans. Consumer Electronics 72(2), 4483–4491, May 2026. DOI 10.1109/TCE.2026.3662053